

開發環境與工具

作業系統: Ubuntu 20.04 LTS (安裝在 VMware Workstation 虛擬機中)

編輯器: Visual Studio Code + Remote-SSH

程式語言: C++11

編譯指令: `g++ -std=c++11 proj1.cpp -pthread -o proj1`

執行方式: 在終端輸入 `./proj1`

實作方法與流程

共用資料結構與排序函式, 這兩個版本只差在參數型態不同。

資料先讀入 `vector<int> data`。為了重複使用同一套排序邏輯, 我實作了:

- `bubbleSort(vector<int> &a, int left, int right)`
針對 `[left, right)` 做氣泡排序。
- `mergeTwoSortedRanges(vector<int> &a, int l, int m, int r)`
合併 `[l, m)` 與 `[m, r)` 兩段已排序區間。
- `mergeSortRecursive(vector<int> &a, int l, int r)`
標準 recursive merge sort, 最後呼叫 `mergeTwoSortedRanges` 處理合併。

需要在 shared memory 這種 `int*` 陣列上操作, 所以另外寫了:

- `bubbleSortArr(int *a, int left, int right)`
- `mergeTwoSortedRangesArr(int *a, int l, int m, int r)`

Method 1: 單行程 Bubble + Merge

流程: 對整個 `[0, n)` 呼叫 `bubbleSort`, 做一次完整 Bubble Sort。再對整段呼叫 `mergeSortRecursive`, 完成最後排序。

Method 2: 單行程、多區段

先定義切段結構: `struct Segment { int l, r; }; // [l, r)`

(1) 切段: 將 $[0, n)$ 切成 K 段, 前面幾段多一個元素, 分布較平均。

(2) 依序 Bubble Sort:

在同一個 process 中, 對每個 Segment 呼叫一次 `bubbleSort(data, l, r)`。

(3) 兩兩 Merge: 使用 `mergeTwoSortedRanges`, 每一輪把相鄰的兩段 $[l, m)$ 、 $[m, r)$ 合併成一段 $[l, r)$, 更新 segment list, 直到只剩一段 $[0, n)$ 。雖然還是單行程, 但因為每一次 Bubble Sort 只處理 N/K 筆資料, 實際速度比 Method 1 好很多。

Method 3: multi-process + shared memory

(1) shared memory 建立與共享

使用 `shmget` 建立一塊大小為 $n * \text{sizeof}(\text{int})$ 的 shared memory, 再以 `shmat` 取得指標 `int *shm`, 並把 data 的內容複製進去。所有 child process 透過 `shmat` 連上同一塊 shm, 達到「process 之間共享資料」的目的。

(2) Bubble Sort: K 個 child process

切段方式與 Method 2 相同。對每一段呼叫 `fork()` 產生 child:

- child 呼叫 `bubbleSortArr(shm, l, r)` 排好自己那一段後 `_exit(0)`;
- 主行程使用 `wait()` 等待 K 個 child 完成。

(3) Merge: child process 合併

Merge 階段也是兩兩合併, 但每一次合併會再 `fork()` 一個 child, 由 child 呼叫 `mergeTwoSortedRangesArr(shm, l, m, r)`, 主行程用 `waitpid()` 等待該 child 結束後更新 segment list。

(4) 回收

最後主行程把 shm 的內容複製回 data, 並呼叫 `shmctl` 刪除 shared memory。

Method 4:

Thread 版不需要 shared memory, 因為同個 process 的 threads 共用同個位址空間。

(1) 參數結構

```
struct BubbleArg {
```

```
vector<int>* data;
int l, r; }; //每個 thread 收到不同的 [l, r)。
```

(2) Bubble Sort 階段

建立 K 個 thread, 執行 bubbleThreadFunc, 在裡面呼叫 bubbleSort(*arg->data, arg->l, arg->r)。主 thread 使用 pthread_join 等待全部完成。

(3) Merge 階段

採用多輪兩兩合併的方式, 每次合併一對 [l, m)、[m, r) 時開一個 thread 執行 mergeTwoSortedRanges, 合併完成後更新 segment list, 直到全部變成一段 [0, n)。和 Method 3 相比, Method 4 不需要管理 shared memory, 程式較簡潔, 但 thread 數過多時也會有排程與 context switch 的成本。

實驗結果與討論

K = {5,300, 1500}	N = 1 萬	N = 10 萬	N = 50 萬	N = 100 萬
方法一	592, 671 586,	58369, 60683 58507,	1502935, 1509470 1456729,	6011957, 7398129 7463391,
方法二	123, 9 10,	11591, 196 79,	295922, 5022 1330,	1216042, 20321 4193,
方法三	32, 262 1113,	2191, 779 1186,	142402, 1787 2519,	268686, 3386 2286,
方法四	83, 153, 1030,	5928, 382 925,	192936, 2794 1311,	599436, 113 2789,

表 1: 實驗記錄表格 (單位: ms)

方法	K = 5	K = 300	K = 1500
方法一	592, 58369, 1502935, 6011957	671, 60683, 1509470, 7398129	586, 58507, 1456729, 7463391
方法二	123, 11591, 295922, 1216042	9, 196, 5022, 20321	10, 79, 1330, 4193
方法三	32, 2191, 142402, 268686	262, 779, 1787, 3386	1113, 1186, 2519, 2286
方法四	83, 5928, 192936, 599436	153, 382, 2794, 113	1030, 925, 1311, 2789

表 2: 實驗記錄表格 (單位: ms)

由表一觀察: 表一是固定三組 K (K = 5、300、1500), 觀察資料筆數 N 從 1 萬、10 萬、50 萬到 100 萬時, 四種方法的時間變化。整體來看, 四個 method 在三組 K 底下的排序關係都很一致: Method 1 最慢, 其次是 Method 2, 而 Method 3 和 Method 4 明顯比較快。

先看 Method 1。在表一的每一欄裡, 只要 N 往右增加一級, 時間就成倍數上升, 尤其是 N=50 萬和 100 萬時, 數字直接進到「好幾萬毫秒」的等級, 幾乎是其他方法的數十倍甚至上百倍。這跟 Bubble Sort 本身是 $O(N^2)$ 完全吻合, 表示只要 N 稍微放大一點, 單行程氣泡排序就會變得非常不可愛。

Method 2 同樣是單行程, 但因為先切成 K 段再 Merge, 每一段要排序的資料比 Method 1 少很多, 所以整體時間有明顯改善。從表一可以看到, 同一個 N 下, Method 2 的數值通常比 Method 1 少一到兩個數量級; 不過當 N 拉到 50 萬、100 萬時, 它的時間還是一路往上衝, 說明「只靠切段」雖然有幫助, 但在大資料量下仍然吃力。

Method 3 與 Method 4 的差異主要出現在大 N 的區域。表一裡可以看到, 當 N 還小 (1 萬、10 萬) 時, 有些情況 Method 4 甚至會接近或略快於 Method 3, 原因是 fork() 與 shared memory 本身有固定的建立成本, 小資料時這些 overhead 比較顯眼; 但當 N 成長到 50 萬、100 萬時, 多核心平行化的效果開始壓過 overhead, Method 3 通常會是四種方法中最漂亮的一個, 而 Method 4 則跟在它後面, 兩者都遠遠領先單行程的 Method 1、2。整體來說, 在固定 K 的前提下, 表一清楚顯示「N 一大, 平行化方法就會大幅勝出」。

由表二觀察：表二則是反過來，固定四組 N（1 萬、10 萬、50 萬、100 萬），比較 K = 5、300、1500 時四種方法的時間變化。這一張表特別用來觀察「資料切成幾段比較好」，以及 process / thread 數量增加到某個程度之後，是否還會帶來效能提升。

首先可以注意到，Method 1 的欄位幾乎不隨 K 改變。同一個 N 下，K 從 5 拉到 1500，Method 1 的時間幾乎沒什麼差異，頂多只是實驗誤差，這是合理的：Method 1 本來就沒有真的用到 K，它就是整段 Bubble + Merge，因此表二可以把 Method 1 視為「對照組」。

對 Method 2 而言，表二呈現的是「一開始變快、之後趨於平緩甚至略回升」的樣子。當 K 從 5 增加到 300 時，每一段資料變短，單次 Bubble Sort 的成本顯著下降，因此時間有明顯改善；但再從 300 增加到 1500 時，雖然理論上每段更短，但需要 Merge 的次數增加、切段與處理的 overhead 變大，所以時間並不會等比例下降，有些 N 下甚至差不多或微微變慢。這說明 Method 2 在 K 上其實存在一個「差不多就好」的區域，不能無限切細。

Method 3 (multi-process) 在表二裡非常適合拿來討論「甜蜜點」的概念。固定某一個 N 來看，K=5 時平行度不足，多核心沒有被完全利用；K=300 時，child process 數量適中，Bubble 部分可以均勻分攤到多顆核心上，同時 Merge 輪數也還可以接受，這時候時間往往是最漂亮的；但 K 拉到 1500，process 數量變得非常多，每次 fork()、wait() 的成本、以及 shared memory 操作和 Merge 動作，都會被放大，導致整體時間不再明顯下降，甚至有些 N 會比 K=300 還慢。從表二來看，可以說 Method 3 的最佳 K 大約落在 300 左右。

Method 4 (multi-thread) 大致上跟 Method 3 的趨勢相似，但沒有那麼極端。由於建立 thread 的成本比 fork() 小很多，固定 N 下，K 從 5 增加到 300 時，時間一樣大幅下降；再從 300 增加到 1500 時，雖然會出現「不再明顯變快甚至持平」的現象，但通常不會像 multi-process 那樣掉太多分。換句話說，對 Method 4 來說，K=300 左右也是一個不錯的折衷點：既能充分平行化，又不至於被排程與同步成本拖垮。

結論：綜合表二可以得到一個很關鍵的結論，K 並不是越大越好。一開始我很單純地以為「切 K 段應該越多越快」，所以特地測了 K = 5、300、1500。結果量測發現，從 5 增加到 300 時，Method 2~4 的效能真的進步非常明顯；但再往上衝到 1500，就不是每一組 N 都會再變快，有些甚至還變慢。這代表只有在某個範圍內，增加 K 才能帶來加速效果，超過之後就會進入「越分工越忙」的區域。直覺上的解釋是：當 K 太多時，process / thread 的建立成本、排程、context switch，加上 Merge 次數變多，這些 overhead 會把原本省下來的益處吃掉。對這次實驗的參數來說，這個比較理想的範圍大約落在 K 介於 5 和 300 之間；超過 300 往 1500 走時，Method 2~4 的時間都不會再有「爆發性的下降」，甚至還會略為回升。整體來說，這次作業不只是讓我把四個版本的排序寫出來，更重要的是讓我用自己的 CPU 和等候時間，親眼體會到 process、thread、演算法設計和 K 值這些原本很抽象的名詞，實際上是怎麼影響執行時間的。